Estimating connective constant of the square lattice

Li Jiaen, A0255750A, e0960420@u.nus.edu

April 22, 2025

1 Introduction

A *self-avoiding walk* (SAW) is a path on a lattice that does not revisit any previously visited site. A self-avoiding walk of length L is denoted as C_L. This quantity serves as a normalization constant for the uniform distribution over all self-avoiding walks of length L. This means that Monte Carlo techniques used to estimate normalization constants can also be applied to estimate C_L.

The number of self-avoiding walks grows exponentially with L, with C_L approximately given by

$$C_L \approx \mu^L$$

where $\mu \approx 2.6$. The goal of this assignment is to estimate μ, or equivalently, λ, where

$$\lambda = \lim_{N\to\infty} \frac{1}{N} \log C_N.$$

Monte Carlo methods provide a simple way to estimate these quantities in low dimensions by generating self-avoiding walks and computing statistical averages to approximate μ.

2 Basic deterministic methods

For small values of L, it is computationally feasible to compute the exact number of self-avoiding walks C_N. These methods provide ground truth values that can be used to validate probabilistic estimates obtained via Monte Carlo simulations.

To compute the exact number of self-avoiding walks (SAWs) of length L, we use a recursive search that explores all valid paths on a 2D square lattice. The procedure is as follows:

1. Begin at the origin $(0, 0)$, marking this as the starting point of the walk.
2. At each step, consider all four possible lattice directions: up, down, left, and right.

3. For each direction, calculate the next coordinate (x', y').
4. If (x', y') has **not** been visited in the current path, extend the walk to that position and continue the search recursively with the remaining length $L - 1$.
5. If the walk has reached length 0, count it as a valid self-avoiding walk.
6. Maintain a set of visited positions to ensure that no lattice site is visited more than once in a given walk.
7. At each recursive level, accumulate the total number of valid SAWs that stem from the current path configuration.

This method exhaustively enumerates all possible SAWs of a given length. While accurate, it becomes computationally infeasible for large L due to the exponential growth in the number of valid paths.

Using the recursive backtracking approach, we can compute the exact values of C_N for small N. Some known values for a square lattice are:

$$
\begin{aligned}
C_1 &= 4, \\
C_2 &= 12, \\
C_3 &= 36, \\
C_4 &= 100, \\
C_5 &= 284, \\
C_6 &= 780, \\
C_7 &= 2172, \\
C_8 &= 5916, \\
C_9 &= 16268, \\
C_{10} &= 44100.
\end{aligned}
$$

3 Basic Monte Carlo I

A direct way to explore self-avoiding walks is to use a simple Monte Carlo simulation. Instead of trying to count all possible self-avoiding walks exactly, we randomly generate many walks and check whether they are self-avoiding. This allows us to get an idea of how common self-avoiding walks are compared to all possible random walks.

The steps for this approach are as follows:

1. Generate a large number of random walks of length L on a square lattice.

2. Check if each generated walk is self-avoiding (i.e., it does not visit any position more than once).
3. Compute the fraction of self-avoiding walks by dividing the number of valid self-avoiding walks by the total number of generated walks.

Since random walks do not deliberately avoid self-intersections, the fraction of self-avoiding walks decreases as L increases. For small values of L, a significant proportion of random walks may still be self-avoiding. However, as L grows, the probability that a randomly generated walk remains self-avoiding becomes very small. This makes this method impractical for large L, as most generated walks will be discarded.

4 Basic Monte Carlo II

In the previous section, we generated random walks and then checked if they were self-avoiding. However, this method is inefficient for large L, as the fraction of self-avoiding walks decreases exponentially with L.

To address this issue, we now implement a Monte Carlo method that directly constructs a self-avoiding walk. Unlike the previous approach, where walks were generated blindly, this method ensures that each step taken avoids self-intersections. This approach allows us to efficiently generate self-avoiding walks of length L.

The algorithm constructs a walk of length L while ensuring self-avoidance at each step. The following steps are taken:

1. Start at the origin $(0, 0)$.
2. At each step, consider the four possible neighboring positions: left, right, up, and down.
3. Select only neighbors that have not been visited before.
4. If at least one valid move exists, randomly choose one and move to that position.
5. If no valid moves exist, the walk remains at its current position for the remaining steps.
6. Repeat until the walk reaches the desired length L.

Since this approach directly generates self-avoiding walks, it is much more efficient than the naive Monte Carlo method.

This approach is not just a method for simulating self-avoiding walks, it can also be used in importance sampling to estimate C_N. Since the algorithm biases the generation towards self-avoiding walks, we can use it as a proposal distribution in an importance sampling framework. This allows us to generate valid self-avoiding walks much more efficiently and obtain better estimates for C_N compared to the naive Monte Carlo approach.

5 Importance Sampling via the Rosenbluth Method

To estimate C_L, the number of self-avoiding walks (SAWs) of length L, we employ a Monte Carlo approach known as the **Rosenbluth method**, which uses importance sampling to efficiently generate and weigh SAWs.

General Importance Sampling Estimator

Let $\pi(x)$ be the target density and $q(x)$ the proposal density. For any function $\varphi(x)$, the expectation under π can be written as:

$$\mathbb{E}_\pi[\varphi(x)] = \int \varphi(x)\pi(x)\,dx = \int \varphi(x)\frac{\pi(x)}{q(x)}q(x)\,dx \approx \frac{1}{N}\sum_{i=1}^{N}\varphi(x_i)\frac{\pi(x_i)}{q(x_i)},$$

where $x_i \sim q(x)$. This allows us to approximate expectations under π using samples from q.

Rosenbluth as an Importance Sampling method

The goal is to estimate C_L, the number of self-avoiding walks (SAWs) of length L. This can be written as:

$$C_L = \sum_{x\in\mathcal{X}} \mathbf{1}_{\text{SAW}}(x),$$

where $\mathcal{X}$ is the set of all possible walks of length L, and $\mathbf{1}_{\text{SAW}}(x)$ is an indicator function that is 1 if x is a self-avoiding walk, and 0 otherwise.

Using importance sampling, we rewrite this sum as an expectation under a proposal distribution $q(x)$:

$$\hat{C}_L = \frac{1}{N}\sum_{i=1}^{N}\varphi(x_i)\cdot\frac{\pi(x_i)}{q(x_i)} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{1}_{\text{SAW}}(x_i)\cdot\frac{1}{q(x_i)}.$$

The proposal distribution $q(x)$ is the probability of generating a particular walk x under the Rosenbluth procedure:

$$q(x) = \prod_{t=1}^{L}\frac{1}{a_t} \quad \Rightarrow \quad \frac{1}{q(x)} = w(x).$$

Thus, the importance sampling estimator becomes:

$$\hat{C}_L = \frac{1}{N}\sum_{i=1}^{N}\frac{\mathbf{1}_{\text{SAW}}(x_i)}{q(x_i)} = \frac{1}{N}\sum_{i=1}^{N}w(x_i).$$

Instead of sampling uniformly from the space of all random walks and rejecting those that are not self-avoiding (a method that becomes exponentially inefficient as L increases), the Rosenbluth method performs biased sampling by constructing self-avoiding walks step-by-step:

1. Generate N SAWs of length L using the Rosenbluth method, which grows walks by randomly choosing from the available non-intersecting directions at each step.
2. Assign an importance weight to each walk:
   (a) If the walk becomes trapped and terminates early (before reaching length L), assign it a weight of 0.
   (b) Otherwise, compute its **Rosenbluth weight**:

$$w(x) = \prod_{t=1}^{L} a_t^{(x)},$$

   where $a_t^{(x)}$ is the number of valid directions available at step t during the construction of walk x.
3. Estimate C_L as the average of these weights.

Note that we do not need to include the indicator function $\mathbf{1}_{\text{SAW}}(x)$ explicitly in the estimator as if a walk becomes non-self-avoiding, it receives a weight of 0 by construction.

Below is a table comparing predicted C_L values (rounded to nearest number) vs actual C_L values given SAWs of length L.

L	Actual C_L	Predicted C_L
1	4	4
2	12	12
3	36	36
4	100	100
5	284	284
6	780	780
7	2172	2173
8	5916	5917
9	16268	16246
10	44100	44086
11	120292	120315
12	324932	324468
13	881500	882140
14	2374444	2374347
15	6416596	6417844

Table 1: Actual vs Predicted C_L

Using the Rosenbluth method to estimate C_L, we can approximate the growth constant μ by computing $C_L^{1/L}$, based on the relationship $C_L \approx \mu^L$. In the figure below, we plot the estimated values of μ derived from Rosenbluth estimates of C_L at varying walk lengths. We observe that the estimates tend to converge toward the current best-known value of μ as L increases. However, at larger values of L, the Rosenbluth method becomes less reliable due to the rapidly decreasing probability of successfully generating valid self-avoiding walks. This results in high variance and potential underestimation of C_L, which in turn affects the accuracy of the μ estimates.

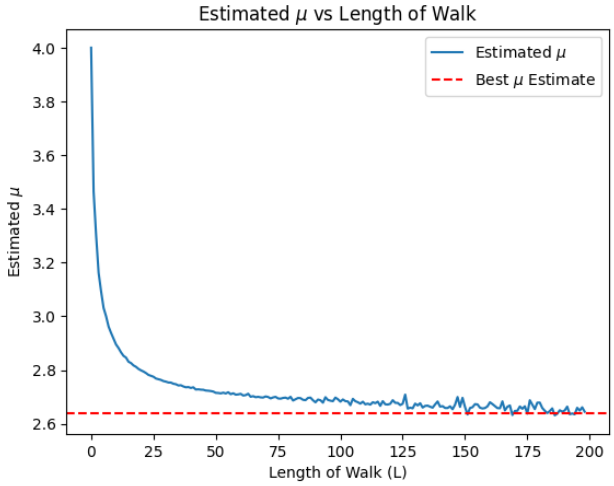


Figure 1: μ estimate derived from Rosenbluth

6 Concatenation Method to estimate C_L

The concatenation method provides an indirect approach to estimating the number of self-avoiding walks (SAWs) of length L, denoted C_L, by leveraging the combinatorial structure of SAWs. The central idea is that if we can generate a sufficiently large collection of SAWs of length L, we can estimate C_{2L} by counting how many concatenations of two SAWs from the collection result in a valid SAW of length $2L$.

This method relies on the assumption that if SAWs of length L are sampled uniformly and independently, the probability that a random pair can be concatenated into a valid SAW of length $2L$ is approximately

$$P = \frac{C_{2L}}{C_L^2}.$$

Rearranging this gives an estimate for C_{2L} as:

$$C_{2L} \approx C_L^2 \cdot P,$$

where P is the observed fraction of valid concatenations. This approach avoids the need to explicitly generate long SAWs, which becomes computationally difficult as L increases.

The following algorithm outlines the steps used in the concatenation method:

1. Initialize an empty list of valid concatenated walks.
2. While the number of valid concatenated SAWs is less than N:
   (a) Generate two independent SAWs w_1 and w_2 of length L.
   (b) Translate w_2 such that its starting point coincides with the endpoint of w_1.
   (c) Concatenate w_1 and the translated w_2 to form a candidate walk of length $2L$.
   (d) If the resulting walk is self-avoiding, add it to the list of valid concatenated SAWs.
3. Let M be the total number of SAW pairs attempted, and N be the number of valid concatenated walks obtained.
4. Estimate the probability P that two SAWs of length L concatenate to form a SAW of length $2L$ by $P = N/M$.
5. Given C_L at small L , estimate C_{2L} by

$$C_{2L} \approx C_L^2 \cdot P.$$

This method can be repeated recursively to estimate $C_{4L}, C_{8L}, \ldots$, providing a scalable approach to estimating the exponential growth of the number of SAWs.

We shall start with C_{20} = 897697164, and slowly build our estimates for $C_{40}, C_{80}, \ldots$, up to C_{640}. Below is a table of predicted C_L values given SAWs of length L.

L	Predicted C_L	Concatenation Probability
20	8.98e+8	-
40	3.18e+17	0.394
80	3.13e+34	0.310
160	2.33e+68	0.237
320	1.06e+136	0.196
640	1.74e+271	0.154

Table 2: Predicted C_L computed by concatenation method

Computing $C_L^{1/L}$ to estimate μ, we can observe a nice convergence towards the current best μ estimate. However, at very large L, the probability of a successful concatenation rapidly decreases, making the method computationally expensive. Furthermore, since the method recursively estimates C_{2L} from previous estimates of C_L, any error in estimating C_L is propagated and amplified in subsequent steps. For instance, an inaccurate estimate of C_{40} will affect the accuracy of C_{80}, which in turn impacts the estimate of C_{160}, and so on. This compounding of error can lead to significant deviations from the true values at large L.

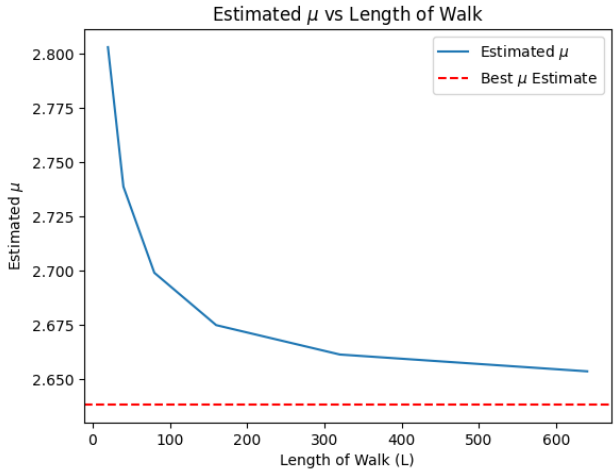


Figure 2: μ estimate derived from concatenation method

7 Estimating μ via Population Growth

To estimate the self-avoiding walk (SAW) growth constant μ, we adopt a recursive population-based approach. The key intuition behind this method is that μ captures the average number of valid directions a self-avoiding walk can extend at each step. While a simple random walk in two dimensions has 4 available directions at each step (yielding 4^L total walks of length L), a SAW is constrained to avoid self-intersections. Thus, μ^L represents the asymptotic growth rate of the number of valid SAWs.

Our method begins with a population of N valid SAWs of small initial length L_0, typically $L_0 = 15$. At each iteration, we extend the walks by one step, estimate μ based on the ratio C_{L+1}/C_L, and repeat until reaching a target length L_{t}. The population size remains fixed at N throughout the process, and resampling is performed at each step based on the number of valid extensions available for each walk.

Our method involves growing a population of N walks iteratively from a small initial length, estimating μ at each step based on the average number of valid self-avoiding extensions. The procedure is as follows:

1. For each walk of length L, count the number of directions that extend the walk while preserving self-avoidance. Compute the average over all N walks. This average is used as an estimate of μ_L at step L.
2. Extend each walk by choosing one of the 3 directions (excluding the direction it came from) uniformly at random.
3. Assign a weight to each extended walk. If the extended walk is self-avoiding, assign weight 1.Otherwise, assign weight 0.

4. Resample N walks from the current population with replacement, using the assigned weights. This ensures that the next iteration proceeds with only self-avoiding walks.

5. Apply the pivot algorithm to the duplicates from the resampled walks to decorrelate and diversify the population. The pivot algorithm randomly selects a pivot point along the walk and applies a lattice symmetry operation, and repeats this process L times. In our case, the symmetry operations are a $90°$ rotation (left or right) or a $180°$ rotation. These symmetry operations ensures ergodicity and can help avoid convergence to a small subset of configurations.[1]

This process is repeated until the desired maximum walk length is reached.

By tracking μ_L across successive values of L, we obtain a sequence of growth constant estimates. These can be further smoothed using running averages or window smoothing to visualize convergence.

Below are the running averages and the window smoothed plot for μ estimates from L = 15 to L = 715.

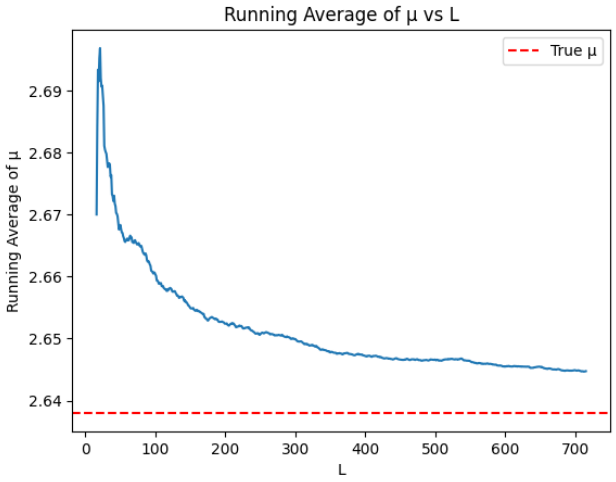


Figure 3: running average of μ

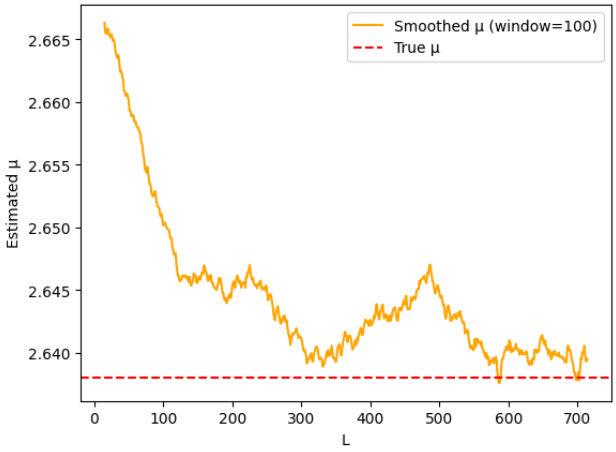


Figure 4: window smoothed plot of μ

We observe that at small L, the estimates of μ tend to be unstable and often overestimate the true value. As L increases, the running average gradually converges toward the current best estimate of μ, indicating improved stability in the population. A similar trend is seen in the window-smoothed plot, which provides a localized average over a fixed window size. However, the window-smoothed estimates exhibit more noticeable fluctuations, especially in the mid-range of L, suggesting that the average number of valid extensions per walk may vary across steps. This increased variation may reflect transient structural biases or limited diversity in the population at certain lengths.

References

[1] Neal Madras and Alan D Sokal. The pivot algorithm: A highly efficient monte carlo method for the self-avoiding walk. *Journal of Statistical Physics*, 50(1):109–186, 1988.